Task 3: Make Your API Safe

Hookshot: A Webhook Delivery Platform (Reliability & Safety)

What is Hookshot and Why Does It Exist?

You are going to build **Hookshot**, a small, real-world version of the webhook systems used by companies like **Stripe, GitHub, and Razorpay**. **GitHub Webhooks:** Sends information to another server when someone pushes code or opens a pull request.

How It Works in Simple Terms

1. **The Developer:** Creates an account, logs in, adds one or more web addresses where they want to receive data, and generates an API key.
2. **The Sender:** The developer's backend sends an event message such as { "type": "invoice.paid" } to Hookshot using the API key.
3. **Hookshot:** Receives the event, makes sure it is not a duplicate, controls the number of requests, and prepares it for reliable delivery.

Step 1: Endpoint Management and User Security

An *endpoint* is simply a web address (URL) where an application wants to receive notifications. In a system with multiple users, **User A must never be able to see, edit, or delete User B's endpoints.** This is called an *ownership check*.

1. What to Do

- Make sure your Task 2 login middleware and protected routes are working and that the logged-in user's ID is available through req.user.id.
- Create a Mongoose model called **Endpoint** containing:
    - userId
    - url
    - active
    - timestamps
- Build four protected Express routes:
    - POST /api/endpoints
    - GET /api/endpoints
    - PATCH /api/endpoints/:id

    - DELETE /api/endpoints/:id
- The GET route should show only the current user's endpoints.
- Check that submitted URLs start with http:// or https://.
- If the URL is invalid, return **400 Bad Request**.

2. How to Check It Works

- Create two separate user accounts: User A and User B.
- Log in as User B and try to update or delete an endpoint belonging to User A.
- You **must** receive a **404 Not Found** response.

Step 2: Event Ingestion

An *event* is a message from another application, such as: "An order was created!"

Your API needs to save this event and find which of the user's active endpoints should receive it later.

Also, make a system diagram to explain the flow of the website (with draw.io or any resource).

1. What to Do

- Create an **Event** Mongoose model containing:
    - userId
    - type
    - data (object)
    - idempotencyKey
    - endpointIds (array of ObjectIds)
    - timestamps
- Create a helper function called getActiveEndpoints(userId) that finds active endpoints belonging to that user.
- Build a protected route: POST /api/events

  It should:

    - ➢ Read the event type and data from the request body.
    - ➢ Read Idempotency-Key from the request headers.
    - ➢ Find the user's active endpoints.
    - ➢ Save the event.
    - ➢ Return **201 Created**.

- Build: GET /api/events
    - ➢ This should show the user's latest **50 events**, with the newest first.

2. How to Check It Works

- Send a test POST /api/events request using Postman or Thunder Client.
- Make sure you include an Idempotency-Key header.
- Call GET /api/events.
- Check that the saved event contains the correct endpointIds.

Step 3: Preventing Duplicate Events

Networks can fail. When an application sends an event and the connection fails, it may send the same event again.

To prevent the same event from being saved twice, the sender includes an Idempotency-Key.

If your server receives a key it has already saved, it should **not save another event**. Instead, it should return the event that was already saved.

Write a 1-page document highlighting the implementation of idempotency and its real-life uses.

1. What to Do

- Add a unique index to your Event schema:

```
eventSchema.index(
  { userId: 1, idempotencyKey: 1 },
  { unique: true }
);
```

- Use try-catch when creating an event.
- If MongoDB returns error 11000, find the existing event using the user ID and idempotency key.
- Return the existing event with **200 OK** instead of creating another one.

2. How to Check It Works

- Send the exact same event **10 times simultaneously**.
- The result should be:

- ➢ 1 saved event
- ➢ 9 safe responses returning the existing event

Step 4: Building the Request Limiter

A request limit means: "A user can make at most N requests within a specific time period."

For example, a user might be allowed 10 requests per minute. This helps stop password guessing and applications sending too many requests.

When the limit is crossed, return: **429 Too Many Requests** and include a Retry-After header.

You will build a **Fixed Window** limiter, where each time period starts with a fresh request count.

1. What to Do

- Create a file called rateLimiter.js.
- Write a function:

  `createLimiter({ limit, windowMs })`

- Use a Map to keep track of requests.
- Check whether an entry exists and whether its time period has expired.
- If the time period has expired, start a new count.
- Increase the count for each request.
- If the limit is crossed, return:

  `{ allowed: false, retryAfterSeconds }`

  Otherwise, return:

  `{ allowed: true }`

- Create Express middleware that uses the limiter, sets the Retry-After header, and returns **429 Too Many Requests** when a request is blocked.

2. How to Check It Works

- Test the limiter using a simple Node.js loop.
- Check that requests are allowed up to the limit and blocked after the limit.

Step 5: Adding Request Limits to Important Routes

Now connect your request limiter to the Express application. Use it to protect important routes from too many requests.

Write a 1-page document explaining Steps 4–5: fixed-window rate limiting, rate-limit keys, middleware, 429 responses, and Retry-After, including why different routes use different limiting strategies.

1. What to Do

- Import your rate limiter into the server.
- Protect the login route using:

  `req.ip + ":" + req.body.email`

- Protect the event route using:

  `req.user.id`

2. How to Check It Works

- Send many requests quickly to the login or event routes.
- Once the limit is crossed, the server should return:
  - ➢ **429 Too Many Requests**
  - ➢ A valid Retry-After header.

Step 6: Implementing Caching

Checking MongoDB for active endpoints every time an event arrives can be slow. A **cache** lets you temporarily store the result so that you do not need to ask MongoDB every time.

The basic process is:

1. Check the cache first.
2. If the data is there, use it.
3. If it is not there, get it from MongoDB.
4. Save the result in the cache.
5. Use it for future requests.

The cached data should expire after a set amount of time. You should also clear the cache immediately when the user's endpoints change.

Write a 1-page document explaining caching and cache invalidation, including cache-aside flow, TTL, cache hits/misses, why caching improves performance, and why invalidation is necessary.

1. What to Do

- Create a file called cache.js.
- Use an in-memory Map.
- Track cache hits and misses.
- Add these functions:

  ```
  get(key)
  set(key, value, ttlMs)
  del(key)
  ```

- Update getActiveEndpoints(userId) so it checks the cache first using a user-specific key such as:

  ```
  "endpoints:" + userId
  ```

- If the data is not in the cache:
  - Query MongoDB.
  - Save the result in the cache.
  - Give it a **5-minute TTL**.
  - Return the result.
- Add cache.del(...) to the endpoint create, update, and delete routes so old cached data is removed immediately.

2. How to Check It Works

- Create a temporary debug route showing cache statistics and database query counts.
- Send multiple events.
- Confirm that database queries drop to **1** while cache hits increase.

Step 7: Integration & Final Testing

Now connect all the features and test Hookshot as one complete system.

1. What to Do

- Connect **authentication → endpoints → events → idempotency → rate limiting → caching**.
- Run tests for:
    a. User ownership and access control
    b. Duplicate events
    c. Rate limiting
    d. Cache hits/misses
    e. Cache invalidation after endpoint changes
- Test edge cases:
    a. Invalid URL
    b. Missing Idempotency-Key
    c. Invalid endpoint ID
    d. Unauthenticated requests
    e. Duplicate events sent simultaneously
- Remove temporary debug routes and console logs.
- Check .env and .gitignore.
- Update the README with setup instructions and API routes.

2. Final Checks

Your API should correctly return:

- 201 for newly created events
- 200 for duplicate events
- 404 for another user's endpoints
- 400 for invalid input
- 429 when rate limits are exceeded
- Retry-After on rate-limit responses

Success condition: *All features work together without duplicate events, unauthorised access, excessive requests, or stale endpoint data.*

FINAL DELIVERABLES

1. Completed Project

Complete the **Hookshot webhook delivery platform** with all the features and integrations described in the task. The final project should be functional, tested, and organized properly.

2. Three One-Page Technical Documents (no AI should be used)

Prepare **three separate one-page documents** at the specified stages of the task.

The **red text in the task specifies what each document should cover**.

For each document, don't just describe the concept. Explain:

- **What the concept/feature does**
- **Why it is needed**
- **How you implemented and used it in Hookshot**
- **How the same concept is relevant to real-world projects / how you could use it in your other projects**

3. System Design Page

Create **one system-design page** that gives a clear overview of your complete Hookshot architecture.

It should include:

- **System architecture / flow**
- **Database models and their relationships**
- **API routes and what each route does**
- **Authentication and authorization flow**
- **Major components/modules**
- **How the different parts of the system interact**
- **The main pages/screens of the application, where applicable**

The diagram should be understandable **without needing to read your source code**.

4. Integration Testing Guides

Create **at least 3 testing guides**, corresponding to major integration stages of your project.

For each testing guide:

- Clearly state **what functionality is being tested**
- Explain **how to set up the test**

- Give the **exact steps/requests** to perform
- Mention the **expected result**
- Include relevant **status codes, headers, or database results**
- Document any **edge cases** you tested

Most importantly, these should be **real testing guides that you actually used while developing and debugging Hookshot**. Do not create tests only for documentation purposes—use them to identify bugs, verify fixes, and make the final system more robust.